ALCATEL-LUCENT ENTERPRISE

ALE Image Forge

Technical Guide for Building Customised Windows 11 Enterprise ISOs with Zero-Touch Deployment

VERSION	AUDIENCE	DOCUMENT TYPE
2.4.1	IT Engineers	Reference Guide

ALE Internal Engineering Documentation

Generated 9 May 2026

Table of Contents

1. Overview & Architecture

ALE Image Forge is a Windows WPF application that takes a stock Windows 11 ISO and produces a **fully customised, bootable enterprise ISO** with:

- A zero-touch unattended installation (no clicks, no prompts)
- Pre-installed applications via a first-boot launcher
- Pre-configured registry, group policies, BitLocker, and security baseline
- Custom wallpaper, drivers, language packs, and deployment scripts
- An auto-generated validation report (TXT + interactive HTML) before the ISO is committed

The output is a single `.iso` file that boots on any Windows 11–capable hardware (with hardware-bypass for TPM/SecureBoot/RAM checks built in) and produces a fully configured, branded, application-loaded machine without any human interaction.

Architecture

Component	What it does
`GoldenISOBuilder.exe`	The build tool itself — WPF wizard that customises the ISO.
`GIBFirstBoot.exe`	A self-contained installer copied into the ISO. Runs on the deployed machine's first login. Installs the bundled apps.
`Autounattend.xml`	Generated XML that drives Windows Setup automatically — no user input.
`SetupComplete.cmd`	Runs as SYSTEM after Windows install completes — handles password, BitLocker scheduling, computer rename, registry tweaks.
`Enable-BitLocker.ps1`	Scheduled task script that enables BitLocker after the TPM stack is ready.

Build Pipeline

Executed by the engine on click of "Build" in Step 8:

1. Validate prerequisites (admin rights, DISM, oscdimg)
2. Clean workspace
3. Extract source ISO
4. Export the selected edition to a single-edition WIM
5. Mount the WIM
6. Inject language packs
7. Inject driver folders
8. Replace wallpaper images (with 4K variants)
9. Inject GIBFirstBoot launcher
10. Stage files for Public Desktop and other locations
11. Stage deployment scripts (with EveryLogin or RunOnce trigger)
12. Remove bloatware (provisioned AppX packages)
13. Enable/disable Windows optional features
14. Apply registry tweaks and group policies via offline hive
15. Generate `Autounattend.xml` + `SetupComplete.cmd` + `Enable-BitLocker.ps1`
16. **Pre-commit validation** — fails the build if any critical check fails
17. Unmount and commit the WIM
18. Rebuild the ISO with `oscdimg` (UEFI + BIOS bootable)
19. Verify SHA-256 of the output

2. Features

2.1 ISO Source with Auto-Detection

What: Drop or browse for a Windows 11 ISO. The app analyses it to extract the available editions, supported architectures, and the boot.wim language.

Advantage: No need to manually know what's in the ISO — the app reads `sources\lang.ini` from the mounted ISO to detect the boot language and automatically narrows the language picker to only languages that exist in the ISO. Prevents the `0x8007000D` boot error caused by language mismatch.

2.2 ISO Boot Language vs Target OS Language

What: Two separate language pickers in Step 1.

Setting	Drives	Why it matters
ISO Boot Language	windowsPE / WinPE — what language Setup itself runs in	Must match the boot.wim language pack. Auto-detected.
Target OS Language	oobeSystem — what language the **deployed Windows** runs in	The locale that users actually see after install.

Advantage: Deploy an English Windows from a French ISO (or any other combination). Prevents the `0x8007000D` boot crash that happens when these are confused with each other.

2.3 Edition Selection

What: After ISO scan, the app shows every edition inside the WIM with size info (Pro, Home, Enterprise, Education, etc.). Select one radio button to lock the build to that edition.

Advantage: Output ISO contains only the chosen edition — smaller file, faster install, no prompt to "Select edition" during Setup. The KMS client key for the selected edition is auto-applied unless an explicit product key is supplied in Step 6.

2.4 Wallpaper Replacement

What: Drop a JPG/PNG. The engine takes ownership of `Windows\Web\Wallpaper\Windows\img0.jpg`, `img19.jpg`, `img20.jpg` (and their 4K variants) and replaces them with the new image.

Advantage: Corporate branding on every deployed desktop without group policy or per-user customisation.

2.5 Staged Apps (First-Boot Installation)

What: Add `.msi` or `.exe` installers with their command-line arguments. The app bundles them into the ISO under `C:\GIB\Installers\`. On the deployed machine's first login, `GIBFirstBoot.exe` reads `apps.json`, installs each app in sequence, tracks progress in `state.json`, and removes itself when complete.

Advantages:

- **Resume-on-reboot:** if the user reboots mid-install, installation continues from where it left off (never re-installs already-completed apps).
- Configurable timeout per app (default 60 minutes, clamped 5–240).
- Configurable accepted exit codes (default `0, 1641, 3010` — 1641 and 3010 are "soft reboot required" codes that aren't real failures).
- No domain join, no MDM enrolment required — apps are baked into the image.

2.6 Staged Files

What: Add arbitrary files with their destination paths in the image (e.g. drop `fr_fr_files.txt` to `Users\Public\Desktop\`).

Advantage: Pre-load licence files, configs, shortcuts, helper documents — anything that needs to be on every deployed machine.

2.7 Deployment Scripts

What: PowerShell scripts staged into `C:\Users\Public\Documents\` with two trigger modes:

Trigger	Behaviour
EveryLogin	Runs at every user logon via the Startup folder
RunOnce	Runs once per user via the RunOnce registry key

Advantage: Per-user configuration (keyboard layout, mapped drives, default printer) that runs without admin rights. The script is in `Public\Documents` so it survives roaming-profile resets.

2.8 Language Packs (.cab)

What: Add downloaded `.cab` files from Microsoft. The engine injects each one with `DISM /Add-Package` while the WIM is mounted.

Advantage: Multi-language deployment from a single ISO. Users can switch language in Settings without a separate download.

2.9 Driver Injection

What: Point to one or more folders containing `.inf` files. Engine injects them all with `DISM /Add-Driver /Recurse`.

Advantage: Hardware-specific drivers (NIC, GPU, chipset, dock) are pre-installed — no out-of-box driver hunt on the deployed machine, no network dependency for initial connectivity.

2.10 Bloatware Removal

What: Tick the provisioned AppX packages you want removed (Candy Crush, Xbox, Bing Weather, Solitaire, etc.). Engine runs `DISM /Remove-ProvisionedAppxPackage` per item.

Advantage: Cleaner Start menu for end users, smaller disk footprint, no compliance issues from non-business apps. Removed BEFORE install, so they never appear for any user.

2.11 BitLocker Auto-Enable

What: Toggle on BitLocker. Choose the drive letter (default `C:`) and whether to save the recovery key (and to what folder).

How it works:

- `Enable-BitLocker.ps1` is staged into the image.
- `SetupComplete.cmd` registers a scheduled task to run it on the NEXT system startup (+2 minute delay) under SYSTEM with highest privileges.
- The script waits for the TPM stack to be ready (up to 5 minutes), then runs `Enable-BitLocker` with `-UsedSpaceOnly -SkipHardwareTest -RecoveryPasswordProtector`.
- Recovery key is saved as `BitlockerPassword_L<MachineSerial>.txt` in the configured folder.
- A run-once marker prevents re-execution on subsequent boots.

Advantage: Full-disk encryption without IT intervention. Recovery key location is dynamic — save it to a shared network path (mapped post-deploy by GPO), to `C:\` for collection by an inventory script, or skip saving entirely for air-gapped environments.

2.12 Defender ATP and SMBv1

What: Two switches: enable Microsoft Defender / disable SMBv1.

Advantage: Security baseline applied without group policy. SMBv1 disablement closes the WannaCry attack surface that's still enabled by default on stock Windows 11.

2.13 System Defaults

Setting	What it does
Dark Mode	Sets system + apps theme to dark
Show File Extensions	Disables Explorer's hide-known-extensions
Show Hidden Files	Off by default
Disable Telemetry	Sets `AllowTelemetry=0` in offline hive
Enable Hyper-V	Enables the Hyper-V optional feature

Advantage: Each toggle is one click but maps to multiple registry keys or DISM operations. Applied to the default user template, so EVERY new user gets these settings.

2.14 Admin Account

What: Set admin username, password, password-never-expires toggle, auto-login toggle. A real-time login preview shows what the OOBE login screen will look like with the wallpaper.

Advantages:

- Password is **base64-encoded** in `SetupComplete.cmd` (never plain-text in the XML or script files).
- Auto-login uses `LogonCount=1` (fires exactly once during OOBE, then self-disables).
- Live password strength meter (4 bars: Very Weak / Weak / Good / Strong) and confirm-password mismatch warning.

2.15 Custom Registry Entries

What: Add arbitrary registry SET or DELETE operations targeting `HKLM\SOFTWARE`, `HKLM\SYSTEM`, or `HKCU\Software`. Engine writes them via `reg.exe` while the offline hives are loaded.

Advantage: Apply settings that don't have GUI toggles. HKCU writes target the default-user template, so they apply to every new user.

2.16 Group Policies (ADMX-based)

What: Browse the ADMX library and pick policies (Enabled / Disabled / Not Configured) with values. Engine resolves the registry mapping and writes to the offline hive.

Advantage: Apply enterprise policies without joining a domain. Useful for golden images delivered to non-domain machines or for compliance baselines applied before the machine reaches the domain.

2.17 OOBE Skip Toggle

What: When enabled (default), generates `Autounattend.xml` for zero-touch install. When disabled, no XML is generated — Windows shows the standard first-boot wizard.

Advantage: Build the same ISO for both unattended bulk deployment and interactive single-machine deployment without changing the rest of the configuration.

2.18 Hardware Compatibility Bypass

What: Built into the windowsPE pass of every generated `Autounattend.xml`:

```
HKLM\SYSTEM\Setup\LabConfig\BypassTPMCheck        = 1
HKLM\SYSTEM\Setup\LabConfig\BypassSecureBootCheck = 1
HKLM\SYSTEM\Setup\LabConfig\BypassRAMCheck        = 1
```

Advantage: Windows 11 installs on machines that don't meet the official minimum requirements — older corporate hardware, lab machines, VMs, etc.

2.19 Upgrade Detection Suppression

What: Adds `<UpgradeData><Upgrade>false</Upgrade><WillShowUI>Never</WillShowUI></UpgradeData>` to the XML, plus sets `DiskConfiguration WillShowUI=Never`.

Advantage: No "Do you want to upgrade?" prompt when re-deploying a machine that already has Windows. Critical for zero-touch on machines being rebuilt.

2.20 BypassNRO (No Microsoft Account)

What: The specialize pass runs:

```
reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\OOBE" /v BypassNRO /d 1
```

Advantage: OOBE doesn't force the user to sign in with a Microsoft account on an offline machine. Critical for enterprise local-admin-only setups.

2.21 Computer Rename

What: Set a computer prefix (e.g. `LAPTOP-`) in Step 6. `SetupComplete.cmd` runs `Rename-Computer` using the prefix + the BIOS serial number, then triggers a single reboot to apply the new name.

Advantage: Every deployed machine ends up with a unique, deterministic, serial-based name — no manual rename, no random GUIDs, easy asset tracking.

2.22 Organization Name and Registered Owner

What: Step 6. Written to `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\RegisteredOrganization` and `\RegisteredOwner`.

Advantage: Shows in `winver.exe` and system properties — useful for inventory, software licensing audits.

2.23 Product Key

What: Optional explicit Windows product key in Step 6. If left blank, the generic KMS client key for the selected edition is used.

Advantage: Suppresses the product-key entry screen during install. KMS keys don't activate Windows — OEM UEFI firmware handles activation on devices with embedded keys, or a KMS server activates them on the corporate network.

2.24 Power Plan, Timezone, OEM Branding

What: Set power plan (Balanced / High Performance), timezone (full Windows timezone list), OEM manufacturer / model / support URL.

Advantage: Pre-configured before first user login. The OEM fields appear in Settings → About → Device specifications and Control Panel's System dialog — useful for branding and tying machines to the asset management system.

2.25 Scheduled Tasks

What: Define tasks with action, trigger (Once / Daily / Weekly / AtLogon / AtStartup), run-as account, highest privileges flag. Engine emits a `schtasks /Create` command for each into `SetupComplete.cmd`.

Advantage: Bundle ongoing maintenance (log cleanup, certificate renewal, weekly defrag) into the image without separate deployment.

2.26 Pre-Commit Validation

What: Before the WIM is committed, the engine runs approximately 60 checks across 7 categories (SESSION, AUTOUNATTEND, SETUPCOMPLETE, STAGED FILES, BITLOCKER, DEPLOYMENT SCRIPTS, REGISTRY). Saves both a plain-text and an interactive HTML report to the Output folder. Any FAIL halts the build with the WIM still mounted so the engineer can investigate.

Advantages:

- Catches misconfigurations BEFORE the 5–10-minute ISO rebuild.
- HTML report has an animated donut chart, plain-English section names ("Drive Encryption" instead of "BITLOCKER"), expandable sections, and a clear "Ready to Deploy" verdict.
- Suitable for handing to managers or auditors who aren't deep-technical.

2.27 Profile Save/Load

What: Save the full Step 1–6 configuration to a `.gibprofile` JSON file. Reload it later to reproduce the exact same build.

Advantage: Standardise builds across the team. Version-control the profile file in Git. Roll back to an older build configuration in seconds.

2.28 Build History

What: Sidebar shows previous builds with date, edition, and success/failure colour dot.

Advantage: Click a previous build to inspect what was used. Reproduce or troubleshoot historical deployments.

2.29 GIBFirstBoot Resume-on-Reboot

What: GIBFirstBoot writes itself to the `RunOnce` registry key BEFORE starting any install. Tracks completed apps in `state.json`. On reboot, reads `state.json`, skips completed apps, resumes from where it left off. Only removes itself from `RunOnce` when ALL apps are confirmed done.

Advantage: Hard reboot, power cut, BSOD during install — none of these break the deployment. The machine resumes correctly on next login.

2.30 Build Settings Persistence

What: Default output and workspace paths, ISO verification, post-build cleanup, WIM compression, sound notifications — saved to `%LOCALAPPDATA%\GoldenISOBuilder\settings.json`.

Advantage: Engineer's preferences persist across app launches and machines (settings file can be copied between workstations).

2.31 First-Boot UI

What: GIBFirstBoot shows a dark-themed window with one row per app, real-time status (Pending / Installing / Done / Failed), and a status bar. On the deployed machine, the technician sees exactly what's happening — no black screen of mystery.

Advantage: During deployment validation, you can watch each app install in real time. Failed apps are visible immediately.

3. The 8-Step Wizard

Step	Page	What you configure
1	Source & Output	Source ISO, ISO boot language (auto-detected), target OS language, Windows edition, architecture, output path, workspace path, output filename pattern
2	Assets	Wallpaper, staged apps (MSI/EXE installers), staged files, language packs (.cab), drivers (folders of .inf), deployment scripts (.ps1)
3	Customizations	Bloatware checklist, BitLocker, Defender ATP, SMBv1, Dark Mode, Show file extensions, Hidden files, Disable telemetry, Hyper-V, group policies
4	Admin Account	Username, password (with strength meter), confirm, auto-login toggle, password-never-expires toggle, live login preview
5	Registry Changes	Custom registry SET/DELETE entries (HKLM\SOFTWARE, HKLM\SYSTEM, HKCU\Software)
6	Advanced Options	Org name, registered owner, computer prefix, product key, Skip OOBE toggle, auto-logon count, power plan, timezone, OEM branding, optional features, scheduled tasks
7	Review	Final summary of the configuration. Read-only.
8	Build	Run the pipeline. Live log + per-step progress. Build report at the end.

4. Use Cases

Each use case below is presented as a real-world scenario you would encounter as an IT engineer. Follow the steps in the wizard, then verify the outcome. Use cases are grouped by deployment category — start at the one closest to your scenario and adapt.

CATEGORY 4.1

Standard Enterprise Deployments

General-purpose office/corporate machines, the bread-and-butter of any IT deployment team.

USE CASE 1

Standard Enterprise Deployment (en-GB ISO → en-US Windows)

Scenario: Bulk-deploy 200 corporate laptops. ALE uses the Microsoft EnglishInternational ISO (en-GB boot) but the end-user OS must be en-US.

Steps

1. **Step 1:** Drop `Win11_25H2_EnglishInternational_x64_v2.iso`
   ISO Boot Language: **en-GB** (auto-detected) — Target OS Language: **en-US** — Edition: **Windows 11 Pro**
2. **Step 2:** Add wallpaper, corporate apps, drivers if needed
3. **Step 3:** Tick bloatware to remove, enable BitLocker, disable SMBv1
4. **Step 4:** Username `Administrator`, strong password, auto-login enabled
5. **Step 5:** Optional custom registry tweaks
6. **Step 6:** Org name `Alcatel Lucent`, computer prefix `L`, timezone `India Standard Time`, leave product key blank
7. **Step 7:** Review → **Step 8:** Build → flash to USB → deploy

---

Outcome: Boots through Windows Setup with no prompts. First login: Administrator auto-logs in, machine renames to `L<SerialNumber>`, reboots once to apply the name, BitLocker schedules itself to run on the next boot, apps install via GIBFirstBoot, machine ends up at the desktop fully configured.

PowerShell needed: None. Engine generates all required scripts.

USE CASE 2

Multi-Language Workforce (FR + CN + EN keyboards, EN display default)

Scenario: ALE has French, Chinese, and Indian staff. Each user needs their preferred keyboard layout, but the default display language must be English so IT support can read error messages.

PowerShell Script — Save as `Keyboard_layout.ps1`

```
$list    = Get-WinUserLanguageList
$changed = $false

foreach ($tag in @('fr-FR', 'zh-CN')) {
    if (-not ($list | Where-Object { $_.LanguageTag -eq $tag })) {
        $list.Add($tag)
        $changed = $true
    }
}

if ($changed) {
    Set-WinUserLanguageList $list -Force -Confirm:$false
    Set-WinUILanguageOverride -Language en-US
}
```

Steps

1. **Step 2 → Deployment Scripts:** Add `Keyboard_layout.ps1` , trigger = **EveryLogin**
2. Build and deploy as Use Case 1

Outcome:

- All users get **English Windows** by default
- French users open Settings → Time & language → Language → change display language to **Français (France)** → reboot → permanently French (script doesn't override on subsequent runs)
- Chinese users do the same with **Chinese (Simplified)**
- Indian users keep English
- Any user can switch keyboards instantly with Win+Space

Why the override is needed: Windows promotes any language with a full pack (fr-FR, zh-CN) over en-US (basic pack only) to the display language position. Without `Set-WinUILanguageOverride` , every machine ends up in French by default.

USE CASE 3

BitLocker with Network Key Backup

Scenario: Compliance requires drive encryption with recovery keys archived to a corporate share. The image is deployed offline; keys must be saved locally then copied to the share by a follow-up inventory script.

Steps

1. **Step 3** → **BitLocker:** Toggle ON — Drive letter: `C:` — Save recovery key: ✓ — Save folder: `C:\`
2. Build and deploy

Follow-up PowerShell (your inventory script)

```
$serial  = (Get-CimInstance Win32_BIOS).SerialNumber.Trim()
$keyFile = "C:\BitlockerPassword_L$serial.txt"
if (Test-Path $keyFile) {
    Copy-Item $keyFile -Destination "\\corp-share\bitlocker-keys\" -Force
    Remove-Item $keyFile -Force   # remove local copy after archival
}
```

Outcome: ~5 minutes after first boot, the scheduled task `EnableBitlocker` fires as SYSTEM with highest privileges. Script waits for TPM ready, enables BitLocker (used-space-only — fast on fresh install), saves the recovery key as `C:\BitlockerPassword_L<Serial>.txt`, creates a marker file, deletes the task, and self-removes. Your follow-up script archives the key to the network share.

USE CASE 4

Image with Pre-Installed Corporate Apps

Scenario: Deploy machines with Acronis Backup, WithSecure AV, Zscaler client, Firefox ESR, Chrome Enterprise, 7-Zip — all installed silently before the user logs in for the first time.

Steps

Step 2 → Staged Apps: Add each app:

App	Type	Args	Timeout / Codes
ABR8.7Workstation.msi	MSI	`/qn /norestart`	30 min
WithSecureOfflineInstaller.msi	MSI	`/qn /norestart`	45 min
Zscaler-windows-4.5.0.296-installer-x64.msi	MSI	`/qn /norestart ENROLL=token`	20 min
Firefox Setup 140.10.1esr.msi	MSI	`/qn /norestart`	Exit: 0,3010
googlechromestandaloneenterprise64.msi	MSI	`/qn /norestart`	15 min
7z2601-x64.exe	EXE	`/S`	10 min

Outcome: First login: GIBFirstBoot window appears showing all 6 apps as "Pending". Each app installs in sequence with live status updates. Progress tracked in `C:\GIB\state.json`. If user reboots mid-install, GIBFirstBoot resumes from where it left off. When done, GIBFirstBoot removes itself from RunOnce, schedules `C:\GIB\` deletion, and closes. Subsequent logins: never runs again. Pre-commit validation confirms all 6 installers are present in `WIM\GIB\Installers\` before the ISO is built.

USE CASE 5

Security and Compliance Baseline

Scenario: STIG / corporate security policy requires telemetry off, SMBv1 disabled, Defender enabled, specific group policies applied, registry-level Spotlight ads disabled.

Steps

1. **Step 3** → **Customizations:** ✓ Disable telemetry — ✓ Disable SMBv1 — ✓ Enable Defender ATP
2. **Step 3** → **Group Policies:** Add ADMX policies:
   - `Turn off all Windows spotlight features` → Enabled
   - `Turn off Spotlight collection on Desktop` → Enabled
3. **Step 5** → **Registry Changes:** Add custom entries:
   - `HKCU\Software\Policies\Microsoft\Windows\CloudContent\DisableSpotlightCollectionOnDesktop` = DWORD 1
   - `HKCU\Software\Policies\Microsoft\Windows\CloudContent\DisableWindowsSpotlightFeatures` = DWORD 1
4. Build

Outcome: Pre-commit validation confirms every registry value is present in the offline hive. HTML report shows each policy in the "System Settings" section with a green tick. Deployed machines comply with the baseline from first boot — no policy-deployment wait.

USE CASE 6

OEM Branded Deployment

Scenario: Customer wants Alcatel-Lucent Enterprise branding on every machine — wallpaper, OEM info, and registered organisation.

Steps

1. **Step 2** → **Wallpaper:** Drop the ALE 4K wallpaper (auto-applied to all variants)
2. **Step 6** → **Advanced:**
   - Org name: `Alcatel-Lucent Enterprise`
   - Registered owner: `IT Department`
   - OEM Manufacturer: `Alcatel-Lucent Enterprise`
   - OEM Model: `Corporate Workstation`
   - OEM Support URL: `https://support.al-enterprise.com`

Outcome: Wallpaper applied to lock screen + desktop on all monitors (4K-aware). `winver.exe` shows the org name. Settings → About shows ALE as the OEM. Control Panel → System shows OEM logo (if you also bundle `OemLogo.bmp` as a staged file).

USE CASE 7

Hardware Bypass for Older Machines

Scenario: Re-image old corporate laptops that lack TPM 2.0 or have only 4 GB RAM. Standard Windows 11 setup refuses to install.

Steps

1. No action needed — TPM, SecureBoot, and RAM checks are bypassed by default in every Autounattend.xml generated by the app
2. Build and deploy normally

Outcome: The three `LabConfig` registry values are written at the start of WinPE. Setup proceeds without the "This PC can't run Windows 11" message. Same image works on TPM 2.0 and non-TPM hardware.

USE CASE 8

Reusable Build Profile

Scenario: The same Pro configuration is built monthly with the latest ISO + latest app installers. Want to ensure consistency.

Steps

1. First build: Configure Steps 1–6, save as `ALE_Pro_Monthly.gibprofile` via the Save Profile button
2. Next month: Click Load Profile, select the file → all settings restored
3. Update the ISO path and any new app versions
4. Build

Outcome: Identical build output every month, parameters version-controllable in Git, no risk of forgotten settings.

USE CASE 9

Single-Machine Interactive Deployment

Scenario: A standalone test machine where IT wants to manually configure the user account at first boot.

Steps

1. **Step 6 → Advanced:** Toggle **Skip OOBE = OFF**
2. Build

Outcome: No Autounattend.xml is generated. The deployed machine runs the standard Windows 11 OOBE wizard — language, region, keyboard, account creation — all asked interactively. All other customisations (apps, wallpaper, drivers, BitLocker) still apply.

USE CASE 10

Driver Pre-Loading for Specialised Hardware

Scenario: Deployment on a Lenovo dock with a USB Ethernet adapter that Windows doesn't recognise out-of-the-box, breaking network at first boot.

Steps

1. Download the Lenovo driver pack — extract to `D:\Drivers\LenovoDock\`
2. **Step 2** → **Drivers:** Add the folder
3. Build

Outcome: Engine runs `DISM /Add-Driver /Driver:"D:\Drivers\LenovoDock" /Recurse` against the WIM. All `.inf` files are injected. After deployment, the dock works the moment Windows boots — network up, GIBFirstBoot can reach licence servers, etc.

USE CASE 11

Scheduled Maintenance Task

Scenario: Every Friday at 22:00, machines should run a temp-file cleanup script.

Steps

1. **Step 6** → **Scheduled Tasks:** Add a task:
   - Name: `Friday Cleanup`
   - Action path: `C:\ProgramData\Scripts\cleanup.ps1`
   - Trigger type: Weekly — Days: Friday — Start time: 22:00
   - Run as: SYSTEM — Run with highest privileges: ✓
2. **Step 2** → **Staged Files:** Add `cleanup.ps1` to `C:\ProgramData\Scripts\`
3. Build

Outcome: Task is registered by `SetupComplete.cmd` on every deployed machine. Cleanup runs unattended every Friday night.

CATEGORY 4.2

Apps, Hardware & Branding

Images that go beyond defaults — pre-loading specific software stacks, injecting hardware drivers, and stamping corporate identity directly into the OS.

USE CASE 12

Developer Workstation — Code-Ready Out of the Box

Scenario: The software engineering team receives new Dell XPS laptops. Each machine must arrive with VS Code, Git, Docker Desktop, Windows Terminal, and WSL2 enabled — fully configured and licence-free — before the developer even logs in.

Steps

1. **Step 2 — Assets:** Add the EXE/MSI installers for VS Code (user-level install with `/VERYSILENT /MERGETASKS=addcontextmenufiles,addcontextmenufolders`), Git (`/SILENT /NORESTART`), and Docker Desktop (`--quiet --accept-license`). Add Windows Terminal from the bundled MSIX.
2. **Step 2 — Features:** Enable `Microsoft-Windows-Subsystem-Linux` and `VirtualMachinePlatform` optional features.
3. **Step 3 — Registry:** Set `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer\Advanced → HideFileExt = 0` and `ShowHidden = 1` (dev-friendly Explorer defaults).
4. **Step 4 — Admin Account:** Set a temporary build admin password; developers will reset via IT on first login.
5. **Step 6 — Build:** Run the pipeline.

PowerShell snippet — VS Code silent install (used as custom args)

```
VSCodeSetup-x64.exe /VERYSILENT /NORESTART /MERGETASKS="!runcode,addcontextmenufiles,addcont
```

Outcome: Developer unboxes the laptop, logs in, opens Windows Terminal — Git, VS Code, and Docker Desktop are already on the PATH. WSL2 is ready to install a distro with one command. Zero IT desk visits.

USE CASE 13

Corporate Wallpaper & OEM Branding Lock-Down

Scenario: The marketing team requires that all company laptops display the official corporate wallpaper and show "Alcatel-Lucent Enterprise" as the manufacturer in System Properties, with the support portal URL. End-users must not be able to replace the wallpaper.

Steps

1. **Step 2 — Wallpaper:** Browse to the approved 4K JPEG brand wallpaper. The engine will overwrite `img0.jpg`, `img19.jpg`, `img20.jpg`, and the 4K variant in the WIM automatically.
2. **Step 5 — Advanced:** Fill in OEM Manufacturer = `Alcatel-Lucent Enterprise`, OEM Model = `Enterprise Workstation`, Support URL = `https://support.al-enterprise.com`.
3. **Step 4 — Registry (custom entries):** Add `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System → Wallpaper = C:\Windows\Web\Wallpaper\Windows\img0.jpg` and `WallpaperStyle = 10` to prevent users changing the wallpaper via Settings.
4. **Step 6 — Build.**

Outcome: Every deployed machine shows the brand wallpaper, "Alcatel-Lucent Enterprise" appears in System Properties, and the wallpaper Settings panel is locked. Consistent corporate identity from day one.

CATEGORY 4.3

Security & Compliance

Images built to satisfy auditors — CIS Benchmark hardening, mandatory BitLocker, USB lockdown, air-gapped offline deployments, and HIPAA/ISO-27001-aligned baselines.

USE CASE 14

Finance Department — Compliance Hardened Image

Scenario: The finance team handles payment data. The security team requires CIS Benchmark Level 1 settings, SMBv1 disabled, telemetry blocked at OS level, BitLocker mandatory, and USB mass storage blocked — all enforced before the user even creates a password.

Steps

1. **Step 3 — Security:** Enable *Disable SMBv1*, *Disable Telemetry*, and *Enable Windows Defender ATP* toggles.
2. **Step 3 — Bloatware:** Remove all consumer apps (Xbox, Solitaire, Weather, etc.) — reduces attack surface.
3. **Step 3 — Group Policy:** Add Computer policy `HKLM\SYSTEM\CurrentControlSet\Services\USBSTOR → Start = 4` (disables USB mass storage driver).
4. **Step 3 — Group Policy:** Add `HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU → NoAutoUpdate = 0, AUOptions = 4` (auto-download and schedule install).
5. **Step 5 — Advanced:** Enable BitLocker. Set the recovery key save path to a secure network share (`\\fileserver\BitLockerKeys\`).
6. **Step 6 — Build.**

Outcome: Every finance machine encrypts its drive at first boot, USB sticks are silently blocked at OS level, and all CIS-aligned registry settings are baked in before first login. Compliance audit evidence is the BitLocker recovery key log and the validation report.

USE CASE 15

Air-Gapped / Offline-Only Deployment

Scenario: A secure R&D lab has no internet access. All software must be bundled inside the ISO. The machine must never attempt Windows Update, must not phone home, and must have all required tools ready without ever touching the network.

Steps

1. **Step 2 — Assets:** Add every required installer as a staged app. Because there is no internet, include offline redistributables (VC++ Redist, .NET Desktop Runtime) that would normally download silently.
2. **Step 3 — Security:** Enable *Disable Telemetry*. This sets the telemetry registry to Security (0) — minimum allowed.
3. **Step 3 — Group Policy:** Add `HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU → NoAutoUpdate = 1` (completely disables Windows Update).
4. **Step 3 — Group Policy:** Add `HKLM\SOFTWARE\Policies\Microsoft\Windows\DataCollection → AllowTelemetry = 0` .
5. **Step 5 — OOBE:** Enable Skip OOBE, disable auto-logon (the lab has dedicated user accounts). Set timezone to the lab's local timezone.
6. **Step 6 — Build.**

Outcome: Machines in the air-gapped lab boot, install all software from the bundled ISO, and never attempt to reach the internet. Windows Update and telemetry are silenced at policy level — even if a user manually visits Settings, the options are greyed out.

CATEGORY 4.4

Specialised Role Images

Tailored builds for specific job functions — helpdesk agents, CI/CD build servers, reception desks — where the OS configuration is part of the role, not an afterthought.

USE CASE 16

Helpdesk Agent Workstation

Scenario: Tier-1 support agents need a standard desktop pre-loaded with remote support tools (TeamViewer Host, AnyDesk, Sysinternals Suite), a custom wallpaper showing the internal helpdesk extension, and shortcuts to the ticketing portal on the Public Desktop.

Steps

1. **Step 2 — Wallpaper:** Use a branded wallpaper with the IT Helpdesk phone number and portal URL embedded as text in the image (created in Photoshop beforehand).
2. **Step 2 — Apps:** Stage TeamViewer Host, AnyDesk, and Sysinternals Suite installer. For Sysinternals (a ZIP), use a deployment script to unzip and add to PATH instead of the app stage.
3. **Step 2 — Public Desktop:** Add a `Helpdesk Portal.url` shortcut file pointing to the ticketing system URL — it lands on every user's desktop on first login.
4. **Step 2 — Deployment Scripts:** Add a script that pins the ticketing portal to the taskbar and sets the default browser to the approved browser.
5. **Step 6 — Build.**

Outcome: A helpdesk agent sits at any machine built from this ISO and immediately has all their tools, the ticketing portal one click away, and the support number visible on the wallpaper — no manual setup required.

USE CASE 17

CI/CD Build Server Image

Scenario: DevOps needs Windows build agents for Azure Pipelines. Each server must have the Windows SDK, .NET 8 SDK, Git, Node.js, and the Azure Pipelines agent installed — and must be configured for optimal performance (no screensaver, high-performance power plan, pagefile on a dedicated drive).

Steps

1. **Step 2 — Apps:** Stage .NET 8 SDK (`/quiet /norestart`), Git (`/SILENT`), Node.js (`/qn /norestart`), and the Azure Pipelines agent ZIP (handled by a deployment script).
2. **Step 2 — Features:** Enable `NetFx3` (some SDK tools require .NET Framework 3.5).
3. **Step 3 — Bloatware:** Remove all consumer / UWP apps — a build server has no interactive users.
4. **Step 3 — Group Policy:** Add `HKLM\SYSTEM\CurrentControlSet\Control\Power → HibernateEnabled = 0` .
5. **Step 5 — Power Plan:** Select *High Performance*.
6. **Step 4 — Auto-Logon:** Enable auto-logon with the build agent service account so the agent starts on boot without human intervention.
7. **Step 2 — Deployment Script:** Add `Register-PipelinesAgent.ps1` that unzips the agent, configures the service with the PAT token, and starts the service.
8. **Step 6 — Build.**

Outcome: Spin up a VM from this ISO, it auto-logs in, the deployment script registers the Azure Pipelines agent, and within 5 minutes the machine appears online in the Azure DevOps agent pool — ready to accept build jobs. No RDP, no manual steps.

CATEGORY 4.5

Shared & Industry-Specific Deployments

Multi-user, kiosk, and vertical-market scenarios — manufacturing floors, education labs, public libraries, and demo machines — where the OS must be opinionated about who can do what.

USE CASE 18

Manufacturing Floor Terminal

Scenario: Industrial PCs on a production line run a bespoke MES (Manufacturing Execution System) application. The OS must auto-log in to a `LineOperator` account, launch the MES on startup, prevent access to anything else, and display the plant manager's OEM branding in System Properties.

Steps

1. **Step 4 — Admin Account:** Username `LineOperator`, strong password, auto-logon enabled, password never expires.
2. **Step 2 — Apps:** Stage the MES client installer (`/S /NORESTART`).
3. **Step 2 — Deployment Script:** Add a startup script that launches the MES in kiosk mode and blocks Task Manager via Group Policy.
4. **Step 3 — Group Policy:** Add `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System → DisableTaskMgr = 1` to prevent operator access to the desktop.
5. **Step 5 — OEM Branding:** Set Manufacturer to the plant name, Model to the machine station ID, Support URL to the internal maintenance portal.
6. **Step 5 — Power Plan:** High Performance. Also add a registry entry to disable screen lock: `HKCU\Control Panel\Desktop → ScreenSaveActive = 0` .
7. **Step 6 — Build.**

Outcome: The terminal boots directly into the MES application. The operator cannot exit to the desktop or access Task Manager. If the machine needs maintenance, the plant manager can identify it via the OEM branding in System Properties.

USE CASE 19

University / Education Lab

Scenario: A university computer lab used by students from 40 countries needs Windows in English as the display language, but with French, Chinese, Arabic, and Spanish available as selectable keyboard layouts. Lab-specific apps (MATLAB, AutoCAD LT, SPSS) must be pre-installed, and students must not be able to install software.

Steps

1. **Step 1 — ISO Language:** Select an *en-GB* source ISO. Set Target OS Language to *en-GB*.
2. **Step 2 — Apps:** Stage MATLAB Runtime installer, AutoCAD LT (`/qn TRANSFORMS=acad.mst`), and SPSS Statistics (`/s /v/qn`).
3. **Step 2 — Deployment Scripts:** Add the `Keyboard_layout.ps1` script (set as a logon script) that adds fr-FR, zh-CN, ar-SA, and es-ES keyboard options without changing the display language.
4. **Step 3 — Group Policy:** Add `HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer → DisableMSI = 1` to prevent student software installs.
5. **Step 3 — Bloatware:** Remove gaming and entertainment apps.
6. **Step 6 — Build.**

Outcome: Students sit down at any lab PC, see Windows in English, and can switch to their preferred keyboard layout from the system tray. MATLAB, AutoCAD, and SPSS are already installed. Students cannot install anything new. Language switches survive reboot.

USE CASE 20

Public Library / Shared Kiosk

Scenario: A public library needs 30 PCs that reset to a clean state between patrons. The machine must auto-log in to a restricted `LibraryGuest` account with no password, only allow approved websites, and look professional with the library's branding. Any files saved to the desktop must be wiped on logout.

Steps

1. **Step 4 — Admin Account:** Username `LibraryAdmin`, strong admin password. Auto-logon to `LibraryGuest` is handled by a separate local account (set up via deployment script on first boot).
2. **Step 2 — Wallpaper:** Library branded wallpaper.
3. **Step 2 — Deployment Scripts:** Add `CreateGuestAccount.ps1` — creates a passwordless `LibraryGuest` account, sets auto-logon to it, and schedules a `CleanupOnLogoff.ps1` task that clears Desktop, Downloads, and Temp on every logoff.
4. **Step 3 — Group Policy:** Add `HKCU\SOFTWARE\Policies\Microsoft\Edge → RestrictSigninToPattern` and `ManagedSearchEngines` to lock the browser to approved sites.
5. **Step 3 — Bloatware:** Remove all non-browser apps. Leave only Edge.
6. **Step 6 — Build.**

Outcome: The library PC boots, auto-logs into the guest account, and shows a clean branded desktop with only the browser. When the patron closes their session, all files are automatically cleaned up. The library IT team can always log in as `LibraryAdmin` for maintenance.

CATEGORY 4.6

Localisation, Virtual & Automation

Advanced scenarios — VDI master images, multi-language international rollouts, multi-edition pipelines, and demo environments — that leverage the full automation depth of the platform.

USE CASE 21

VDI Master Image (Citrix / VMware Horizon)

Scenario: The virtualisation team needs a Windows 11 master image for Citrix Virtual Apps & Desktops. The image must be stripped of all unnecessary services and animations to minimise CPU/RAM per session, have the pagefile set to a fixed size, and include the Citrix VDA agent pre-installed.

Steps

1. **Step 3 — Bloatware:** Remove all UWP apps — virtual desktop users access apps via published desktops, not the Start menu.
2. **Step 3 — Features:** Disable `WindowsMediaPlayer`, `PrintToPDFServices`, and `XPS-Viewer`.
3. **Step 3 — Registry (VDI optimisations):** Add the following entries:
   - `HKLM\SYSTEM\CurrentControlSet\Services\WSearch → Start = 4` (disable Windows Search indexer)
   - `HKLM\SYSTEM\CurrentControlSet\Services\SysMain → Start = 4` (disable Superfetch)
   - `HKLM\SYSTEM\CurrentControlSet\Services\DiagTrack → Start = 4` (disable Connected User Experiences)
   - `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer\VisualEffects → VisualFXSetting = 2` (adjust for best performance)
4. **Step 2 — Apps:** Stage Citrix VDA installer (`/quiet /norestart /components VDA /controllers "ddc.corp.local" /enable_hdx_ports /optimize`).
5. **Step 5 — Power Plan:** High Performance.
6. **Step 5 — Advanced:** Set a fixed pagefile size via deployment script to avoid dynamic expansion under load.
7. **Step 6 — Build.**

Outcome: The VDI master image hosts 15–20% more concurrent sessions than an unoptimised baseline. Citrix VDA registers automatically on first boot. The Citrix administrator can immediately take a snapshot and publish it as a machine catalogue.

USE CASE 22

Multi-Language International Office Rollout

Scenario: A global rollout covering the UK office (English), Paris office (French display language), and Shanghai office (Chinese display language). The IT team wants one base process, three ISOs — built from the same wizard profile but with different Target OS Language settings each time.

Steps (repeated 3 times, changing only the language setting)

1. **Step 1 — ISO Boot Language:** Use the same *en-GB* multi-language source ISO (contains en-GB, fr-FR, zh-CN language packs).
2. **Step 1 — Target OS Language:** Set to *en-GB* for UK, *fr-FR* for Paris, *zh-CN* for Shanghai.
3. **Step 2–5:** Identical settings (apps, security, admin account) across all three — load the same `.gibprofile` each time and change only the Target OS Language.
4. **Step 5 — Timezone:** *GMT Standard Time* for UK, *Romance Standard Time* for Paris, *China Standard Time* for Shanghai.
5. **Step 5 — Output Filename:** Use a descriptive name such as `GoldenImage_Pro_en-GB`, `GoldenImage_Pro_fr-FR`, `GoldenImage_Pro_zh-CN`.
6. **Step 6 — Build** each variant.

Outcome: Three ISOs — each identical in apps and security posture, but with the right display language and timezone for each office. Machines in Paris boot in French; in Shanghai, in Chinese. One profile, three builds, zero manual language configuration after deployment.

USE CASE 23

Product Demo Machine — Always-Ready Showcase

Scenario: The pre-sales team takes demo laptops to customer meetings. The machine must boot to a polished desktop showing the product demo environment, with all demo apps open at startup, a clean brand wallpaper, no distracting notifications or update prompts, and a demo user account with a blank memorable password.

Steps

1. **Step 4 — Admin Account:** Username `Demo`, password `demo1234` (memorable for the sales team), auto-logon enabled, password never expires.
2. **Step 2 — Wallpaper:** High-resolution product marketing wallpaper.
3. **Step 2 — Apps:** Stage all demo application installers.
4. **Step 2 — Deployment Scripts:** Add `SetupDemoEnv.ps1` that pins demo apps to the taskbar, disables Action Center, and adds demo apps to the Startup folder.
5. **Step 3 — Security:** Enable *Disable Telemetry* (no pop-ups about data collection). Disable Windows Update via Group Policy to prevent mid-demo update reboots.
6. **Step 3 — Bloatware:** Remove all non-demo apps to keep the Start menu clean.
7. **Step 5 — OOBE:** Skip OOBE, enable auto-logon.
8. **Step 6 — Build.**

Outcome: The sales engineer powers on the laptop, it boots directly to a polished desktop with all demo apps ready, no update nags, no telemetry consent dialogs. The customer sees a professional, product-focused environment within 60 seconds of power-on.

USE CASE 24

Automated Multi-Edition Pipeline (Home / Pro / Enterprise)

Scenario: A large organisation licences Windows in three tiers: Home for frontline workers, Pro for knowledge workers, and Enterprise for power users and IT staff. The IT team needs three ISO variants from the same source, each exporting a different edition, but sharing the same hardening, apps, and admin configuration.

Steps (one saved profile, three runs)

1. Build the full wizard configuration once — security, apps, registry, admin account, all shared settings.
2. Save the profile as `Base_Config.gibprofile` using *File → Save Profile*.
3. **Run 1 — Home:** Load `Base_Config.gibprofile`. Step 1 → Target Edition = *Home*. Step 5 → Output filename = `GoldenImage_Home_v1`. Build.
4. **Run 2 — Pro:** Load `Base_Config.gibprofile`. Step 1 → Target Edition = *Pro*. Step 5 → Output filename = `GoldenImage_Pro_v1`. Build.
5. **Run 3 — Enterprise:** Load `Base_Config.gibprofile`. Step 1 → Target Edition = *Education* or *Enterprise*. Step 5 → Output filename = `GoldenImage_Ent_v1`. Add additional Enterprise-only group policies (AppLocker, DirectAccess) before building.

Pro tip

Use the *Quick Build* button on the Home page between runs — it re-runs the last profile without going through all 8 wizard steps. Change only the edition and filename, then click Build.

Outcome: Three ISO files, one source, one IT team. Each edition inherits the full security baseline and app stack. Home machines get standard hardening; Enterprise machines get additional policies on top. Consistent, auditable, and fast to re-build when the base config changes.

5. Validation Report Interpretation

After every build, two reports are saved to your output folder:

- `ValidationReport_<timestamp>.txt` — plain text for archives or pasting into tickets
- `ValidationReport_<timestamp>.html` — interactive HTML with donut chart, expandable sections, colour-coded status

Reading the HTML report

Element	Meaning
Health Score (gold ring)	% of checks that passed. 100% = nothing to action.
Stat cards (green/amber/red)	Pass / Warn / Fail counts
Verdict banner	"Ready to Deploy" (green) / warnings (amber) / "Build Halted" (red, pulsing)
Section cards	Plain English names: "Windows Setup Script", "Drive Encryption", "Bundled Software & Files", etc.
Each row	Green ✓ / Amber ! / Red ✕ with the technical detail underneath in monospace

Sections with failures or warnings expand automatically; passed sections are collapsed for brevity. Click a section header to toggle.

Common warnings that are safe to ignore

Warning	Why it's safe
`Keyboard_layout.ps1 referenced` / `not in Startup folder`	By design — the script runs from `Public\Documents` via EveryLogin trigger
`Custom SET ... data check inconclusive`	The registry key IS present (value confirmed in the output) — validator just can't compare because the expected value was empty string
`img20.jpg not present in this Windows build, skipping`	Not all Win11 builds ship every wallpaper variant — `img0.jpg` and `img19.jpg` were replaced successfully

6. Quick Reference — Key File Locations

What	Where
Application	`...\GoldenISOBuilder\bin\x64\Release\net8.0-windows10.0.17763.0\GoldenISOBuilder.exe`
App settings	`%LOCALAPPDATA%\GoldenISOBuilder\settings.json`
Crash log	`%LOCALAPPDATA%\GoldenISOBuilder\crash.log`
Build log (per build)	`<OutputPath>\build-<timestamp>.log`
Validation reports	`<OutputPath>\ValidationReport_<timestamp>.txt` and `.html`
GIBFirstBoot on deployed machine	`C:\GIB\GIBFirstBoot.exe`
App manifest	`C:\GIB\apps.json`
Install state	`C:\GIB\state.json`
BitLocker script	`C:\Windows\Setup\Scripts\Enable-BitLocker.ps1`
BitLocker log	`C:\Windows\Setup\Logs\Bitlocker.log`
BitLocker recovery key	`<configured folder>\BitlockerPassword_L<Serial>.txt`
Deployment scripts	`C:\Users\Public\Documents\*.ps1`
Autounattend (on ISO root)	`<ISO>\Autounattend.xml`
Unattend (inside WIM)	`<C:>\Windows\Panther\unattend.xml`
SetupComplete log	`C:\Windows\Setup\Scripts\setupcomplete.log`

7. Troubleshooting Checklist

Symptom	First check
`0x8007000D` at boot	Step 1 ISO Boot Language ≠ ISO's boot.wim language. Should be auto-detected — verify the banner shows "Auto-detected from ISO".
"Did you try to upgrade?" prompt	Already suppressed in current builds via `<UpgradeData>` + `DiskConfiguration WillShowUI=Never`. If still appearing, the ISO was built with an older version — rebuild.
BitLocker scheduled task never runs	Check `/DELAY` value — must be `0000:02` (HHHH:MM = 2 minutes), not `0002:00` (= 2 hours). Already correct in current builds.
BitLocker recovery key missing	Check `C:\Windows\Setup\Logs\Bitlocker.log` on the deployed machine. TPM may not have become ready within 5 minutes.
Apps not installing on first boot	Verify `C:\GIB\GIBFirstBoot.exe` exists and `apps.json` lists the apps. Check `C:\GIB\state.json` for `Failed[]` entries.
GIBFirstBoot didn't run after a mid-install reboot	Check `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce\GIBFirstBoot`. If absent, the previous run was killed before re-registering — already fixed in current builds (registers at start, removes only on full completion).
Display language becomes French unexpectedly	The keyboard deployment script must include `Set-WinUILanguageOverride -Language en-US` when adding languages — see Use Case 2.
Build halts with "GIBFirstBoot.exe not found"	The MSBuild target `PublishAndCopyGIBFirstBoot` didn't run. Re-publish the app.

8. Build Commands (for Developers)

Working directory: `C:\Users\<user>\Downloads\ALE ISO Creator\GoldenISOBuilder\`

Build only — produces framework-dependent output

```
dotnet build -c Release -p:Platform=x64 --nologo
```

Publish — produces single-file self-contained exe for distribution

```
dotnet publish -c Release -p:Platform=x64 -r win-x64 --self-contained true -p:PublishSingleFile=tr
```

Build folder requires .NET 8 installed on the machine. Publish folder is self-contained (~174 MB), bundles the full .NET 8 runtime, runs on any Windows machine without .NET.

9. Runtime Requirements

Where	What's needed
Build machine	Windows 10/11, .NET 8 SDK (for dev) or .NET 8 runtime (for using the published exe), Windows ADK Deployment Tools (for `oscdimg.exe`), administrator rights, ~25 GB free disk for workspace
Deployed machine	Any Windows 11–capable hardware (TPM/SecureBoot/RAM bypassed automatically). For the BitLocker scheduled task: TPM 2.0 must be present and enabled in firmware.

Generated for ALE Image Forge v2.4.1 — Engineering Documentation